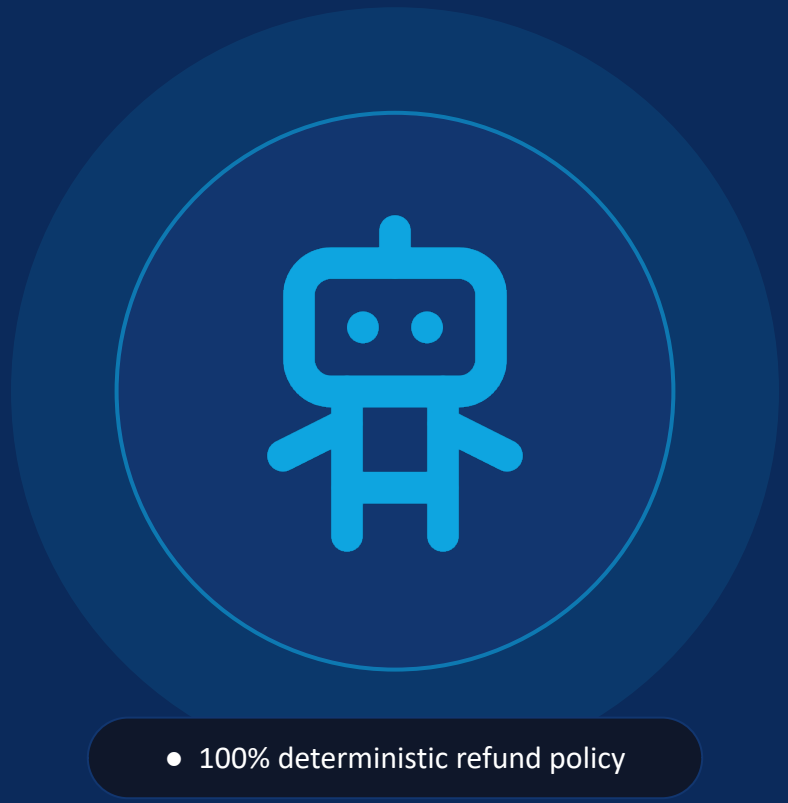
PAYTM HACKATHON 2026 · TRACK 3: AUTONOMOUS AI TEAMMATES

Resolv

The AI support teammate that **resolves** failed UPI payments end to end, instead of just talking about them



- 100% deterministic refund policy

PROBLEM STATEMENT Build AI teammates that don't just respond; they get the job done

TEAM NAME 4-Goats

TEAM MEMBERS Paras Patil · Swastik Athanikar
Nitish Shinde · Sanchit Tawde

Understands · Acts · Verifies · Escalates

GitHub: github.com/Paras131206/PAYTM_hackathon_2026

Problem statement

OFFICIAL PROBLEM STATEMENT

"How might we build an AI teammate that can independently handle a customer-service or sales workflow end-to-end — resolving issues, taking actions across systems, and escalating to humans only when needed?"

Our focus: customer service for failed and disputed UPI payments, the most common payments-support ticket.

"Money debited, payment failed"

Every ticket needs the same manual checks: payment switch, ledger, refund policy, then a reply. Customers wait and worry.

Chatbots answer, they don't act

Today's bots can explain a refund but cannot issue it, verify it or close the ticket, so a human still does the work.

Letting AI move money is risky

An LLM can invent an amount, obey a prompt injection, or claim a refund that never happened.

T+1

₹100/day

Slow resolution has a real cost. RBI requires a failed UPI debit to be reversed by T+1. After that, the customer must be compensated ₹100 for every day of delay.

Source: RBI circular on TAT harmonisation and compensation for failed transactions, 20 Sep 2019

Proposed solution: Resolv

Resolv AI teammate

Core idea: the AI decides what to do, fixed rules decide what is allowed, tools do it, and the ledger proves it.

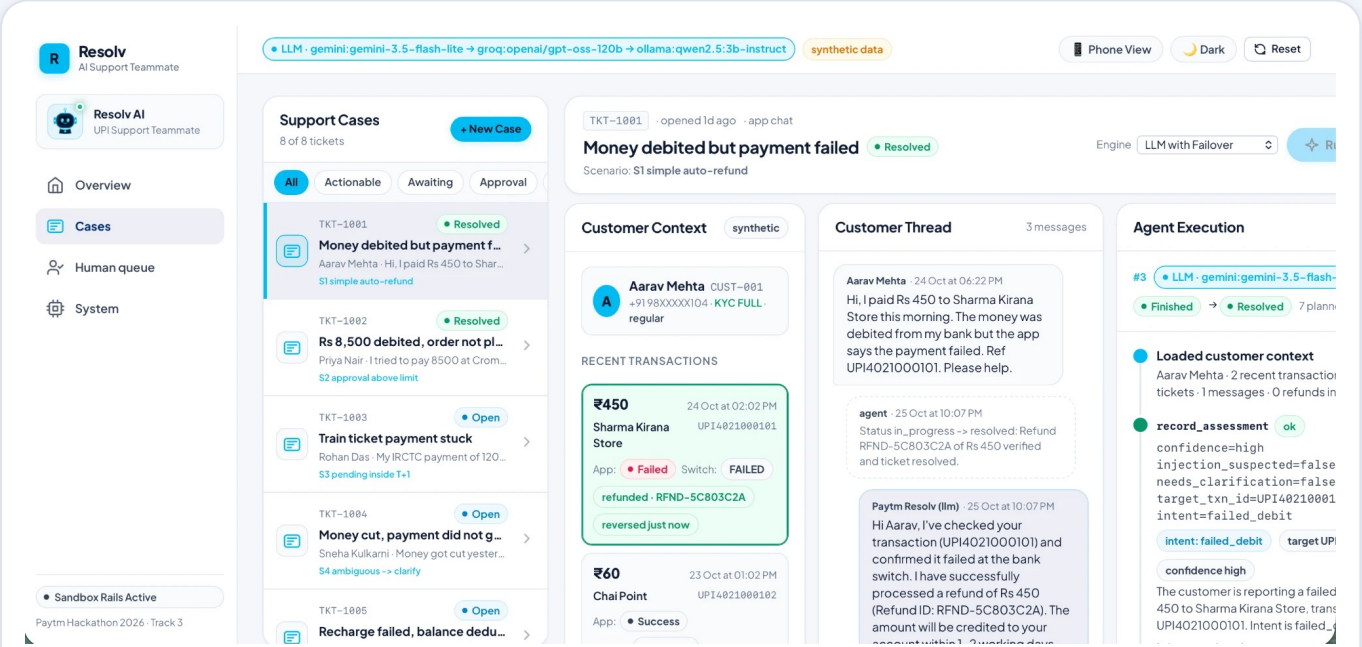
Understands
Profile, recent payments, past tickets and the chat.

Decides
Intent and which payment; asks once if unsure.

Acts
Checks the switch, refunds, updates the ticket, replies.

Verifies
Confirms the refund in the ledger before saying so.

Escalates
Big refunds, disputes, outages, tricks go to a human.



Working prototype: the case workspace. The ₹450 payment shows “refunded” after a verified ledger check.

WHO IT'S FOR

- **UPI app support teams**
- **Payment aggregators**
- **Bank service desks**
- **Merchant operations**

End-to-end workflow

Resolv AI teammate

1 — Customer message
App chat: “₹450 debited, payment failed”

2 — Load context
Profile, last 10 payments, past tickets, policies

3 — Assess
Intent + which payment + manipulation check

4 — Gather evidence
Ask the payment switch: SUCCESS / FAILED / PENDING

5 — Policy decision
Fixed rules: auto-refund, approve, wait or block

6 — Act
Refund (amount from ledger), update ticket, reply

7 — Verify
Re-read the ledger; no claim before proof

8 — Close or hand off
Resolve + notify, or human queue with evidence

OUTCOMES

- Auto-resolved
- Human approval (> ₹2,000 / flagged)
- Wait (pending inside T+1)
- Escalate (dispute · outage · tricks)

Technology & tech stack

Resolv AI teammate

SYSTEM ARCHITECTURE

React UI
Cases, timeline, human queue, metrics

→

FastAPI
Typed API, background agent runs

→

Orchestrator
Turn budget, audit, outcome from state

→

LLM chain
Gemini → Groq → Ollama failover

→

Policy + tools
Rules engine, 10 validated tools

→

Sandbox + SQLite
Switch, refund ledger, audit trail

Tech stack

- **Python 3.14 · FastAPI · Pydantic** backend, tool schemas
- **React 19 · TypeScript · Vite** operations UI
- **Gemini · Groq · Ollama (local)** LLM providers, free tiers
- **SQLite** tickets, ledger, audit events
- **pytest (95 tests) · GitHub** quality and versioning

Why this stack

- **FastAPI + Pydantic:** every tool call is validated against a typed schema before it runs.
- **Three LLM providers:** if one hits a quota or outage, the run switches mid-ticket. Ollama works offline.
- **SQLite:** zero setup, and every decision, tool call and result is stored for audit.
- **React + TypeScript:** a typed UI that shows real backend state, never mock activity.

USP: what makes Resolv different

Resolv AI teammate

Acts, not just answers

Issues the refund, updates the ticket and replies, in one run.

Policy lives outside the AI

Fixed rules decide refunds; amounts come from the ledger, never the model.

Verify before claiming

Can't say “refunded” or close a ticket until the ledger confirms it.

Humans where it matters

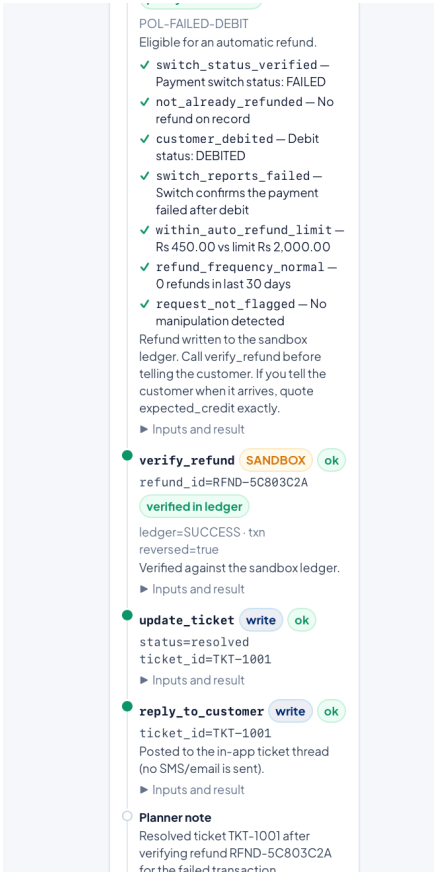
Approval queue shows evidence and policy checks for every handoff.

Never fakes an answer

Gemini → Groq → Ollama failover; if all fail, a human takes over.

Audit trail + injection guard

Every step is logged; “admin mode” tricks are flagged and blocked.



Real policy checks and ledger proof in the live timeline

Impact & benefits

Resolv AI teammate

8 / 8

demo scenarios handled correctly

100%

of refunds verified in the ledger before the customer was told

14–83 s

live LLM run, from complaint to resolved ticket

95

automated tests: policy, tools, failover, API

Prototype measurements on synthetic demo data (sandboxed payment rails), not business results.

Customers

- Money back faster, inside the T+1 window
- Clear, honest status updates in the app chat

Support teams

- Repetitive lookups and refunds are handled for them
- Every handoff arrives with evidence and policy checks

The business

- Avoids ₹100/day delay compensation
- Consistent policy, no wrong or duplicate refunds, audit-ready

Business model

Resolv AI teammate

B2B SaaS · PROPOSED

Customers

- UPI apps & payment service providers
- Payment aggregators
- Banks' digital service desks
- High-volume merchants

Value

- Resolves routine payment tickets end to end
- Keeps humans for exceptions
- Stays inside RBI timelines
- Full audit trail

Revenue

- Platform fee per support queue
- Outcome fee per verified, auto-resolved ticket
- Enterprise tier: on-prem / local LLM, custom policies, SLA

Costs

- LLM usage: low, with free-tier and local models in the failover chain
- Hosting
- Integration with switch and refund APIs

GO-TO-MARKET

1 **Pilot** One support queue: failed UPI debits → 2 **Expand** Wallet, merchant settlement, KYC tickets → 3 **Scale** Offer to other PSPs, banks and merchants

Outcome-based pricing: customers pay most when tickets are actually resolved, matching the brief's "measurable outcomes".

Future scope & references

Resolv AI teammate

Real integrations

Bank / PSP sandbox APIs for switch status and refunds

Multilingual & voice

Hindi and regional languages, voice notes

Compensation automation

Auto-credit ₹100/day when T+1 is missed

Roles & security

Login, approval roles, per-team policies

Measured pilot

Resolution time vs today's baseline, A/B tested

References

- RBI: TAT & compensation for failed transactions (2019)
- ReAct: reasoning + acting in language models (Yao et al.)
- Gemini API: function calling
- Groq API documentation

Links

GitHub repository
Paras131206/PAYTM_hackathon_2026

Demo video
[add link]